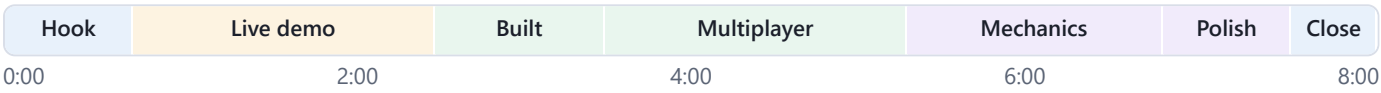
STRATHHACK 2026 · TEAM 22

Space Salvage

Presentation pack: run-of-show, word-for-word script, demo run sheet, Q&A answers and speaker cue cards. 8 minutes of talk + 2 minutes of questions.

Run of show



Time	Segment	Speaker	On screen
0:00 – 0:45	1. Hook & pitch	Speaker A	Game title / menu screen
0:45 – 2:30	2. Live demo	Speaker B + players	The game, played live on 2–3 laptops
2:30 – 3:30	3. How it's built	Speaker C	Project files / architecture diagram (study guide, Figure 1)
3:30 – 5:15	4. Multiplayer	Speaker C	`network_manager.gd`, `loot.gd`
5:15 – 6:45	5. Communication mechanics	Speaker D	`tethers.gd`, `player.gd`, `heavy_relic.gd`
6:45 – 7:30	6. Polish & testing	Speaker D	Nebula shader / bot test running
7:30 – 8:00	7. Challenges & close	Speaker A	Win screen with confetti (frozen from the demo)
8:00 – 10:00	Questions	Whoever owns the topic	Game left running

Speakers A–D are roles: assign one person to each. With 3 people, merge A and B. With 2, one person does A + B + D and the other does C. Each script is about 140 words per minute, a relaxed speaking pace.

0 Before you present (do this 15 minutes early)

Laptops & network

☐ Everyone on **the same Wi-Fi**, or better, one **phone hotspot** (venue Wi-Fi often blocks laptop-to-laptop traffic).

☐ Latest build on every laptop (`git pull` on main, or the exported `SpaceSalvage.exe`).

☐ Host laptop: Godot allowed through the **Windows Firewall**, ticked for **Private and Public**.

☐ Do one full test join with the **3-number code** on the exact network you'll present on.

☐ Laptops plugged in; notifications and sleep turned off.

Sound & screen

☐ The presenting laptop is connected to the projector, and its volume is **up** (the music and the snap sound sell the demo).

☐ Other laptops **muted** (avoids echo).

☐ Window large / full screen on the projector laptop.

Pre-load the demo

☐ **Laptop 1 (projector), host:** start the game with the launch argument `-- --money=6000` so the team starts rich enough to buy all 5 pod parts for the finale. In the editor: Debug › Customize Run Instances › Main Run Args. For the exe: `SpaceSalvage.exe -- --money=6000`. Menu open, name entered, ready to click **Host Game**.

☐ **Laptops 2 & 3:** join code typed in, ready to click **Join Game**.

☐ Code open in the editor on the projector laptop, in tabs: `network_manager.gd`, `loot.gd`, `tethers.gd`, `heavy_relic.gd`, `nebula.gdshader`.

Roles during the demo

Who	Plays as
Speaker B	**Host + diver** on the projector laptop, narrates
Player 2	**Radar operator**: presses R, calls out loudly
Player 3	Diver, roped to Speaker B

> **Rehearse once with a stopwatch.** The demo is where talks overrun. If you're behind at 2:30, skip straight to the pod launch.

1 The script

The blue boxes are what to **say**, word for word or close to it. ACTION tags are what to do. "If you forget" lists the bullets to fall back on.

1. Hook & pitch Speaker A 0:00 – 0:45

On screen: **the Space Salvage menu** on the projector.

> "Hi, we're Team 22, and this is **Space Salvage**, a co-op game where **nobody can succeed alone**.
>
> Think *Dave the Diver*, but in space, seen side-on like *Terraria*. Your crew drifts out from a space station in zero gravity, grabs gold that's worth more the further out you go, and has to get home before the air runs out. You sell your salvage, buy upgrades, and repair an escape pod to get home.
>
> The theme was **Discord Games**, where communication is the mechanic. So we built three systems that make you *have* to talk: **darkness with a radar**, **ropes that tie you together**, and **relics that take two people to carry**. Let us show you."

If you forget: co-op salvage in space · air runs out · sell → upgrade → escape pod · theme = communication · radar, ropes, relics.

2. Live demo Speaker B + 2 players 0:45 – 2:30

On screen: **the game, from the projector laptop's diver**, so the audience sees the dark, the rope and the snap. The radar operator plays on another laptop and calls out loudly.

> Click Host Game "I'm hosting. My friends join by typing just these three numbers in the corner, the **join code**." Players 2 & 3 click Join
>
> "We start in the station. Inside there's gravity, so it plays like a platformer. Let me show you the **radar** first." Press R "Whoever stays home sees everything: all the gold, every teammate, and how much air they've got." Press R again "Tonight our radar operator is staying home, and I'm diving." Player 2 presses R
>
> Walk out the airlock "Outside it's **dark**: I can only see what's in my helmet lamp, and I'm **roped** to my partner. So I'm completely relying on the voice from the station."
> Radar operator calls out live: "Gold bar up and to your left! You're at 40% air!" Follow it, grab the gold
>
> "That's the game: we're blind, the radar is our eyes, and we have to talk."
> Both divers grab the glowing relic "This relic is worth $500, but it only moves when **both** of us hold on, and it goes to the point between us, so we have to agree on a direction."
>
> "And if we **stop listening** to each other..." Fly apart until the rope snaps "...the rope snaps, we **both** black out and lose everything." Wait for the laugh
>
> Back in the station, press 3 five times "We've given ourselves some money so we can show you the ending. When the team fits all five repair parts, the escape pod launches, and you made it home."

If you forget: join code · show the radar (R) then dive · dark + roped · radar operator calls out loot and air · relic needs 2 · rope snap · back inside, 3 ×5 → pod launch.

> **If the networking fails live:** don't debug on stage. Say "*live Wi-Fi demos, eh*", play solo on the projector laptop (you'll be roped to the station), show the dark, the radar (R inside), then press 3 five times in the station for the pod launch. It still tells the story.

3. How it's built Speaker C 2:30 – 3:30

On screen: **the project's file list** or the architecture diagram (study guide, Figure 1).

> "We built it in **Godot 4.7** with GDScript: about **2,600 lines across 15 scripts**, plus one shader.
>
> Two scripts are **autoloads**, meaning they're always running: `NetworkManager` handles connecting players, and `GameState` holds the team's shared money, upgrades and repair parts.
>
> The menu loads the **world scene**, and `world.gd` builds everything else: the players, 160 pieces of loot, three relics, the station, the HUD and the background.
>
> One decision that helped us a lot: we build almost all the UI and objects **in code** instead of the editor. Godot scene files merge badly in git, so this kept them tiny, and the whole team could work in parallel without conflicts."

If you forget: Godot 4.7 · 2,600 lines · 2 autoloads (NetworkManager, GameState) · world.gd builds everything · UI in code → no merge conflicts.

4. How the multiplayer works Speaker C 3:30 – 5:15

On screen: **`network_manager.gd`** (code_to_ip), then **`loot.gd`** (request_collect / confirm).

> "Multiplayer was the core of the project, so here's how it works.
>
> One player **hosts**: their game becomes the server, using ENet on UDP port 9999, for up to eight players. Typing IP addresses is painful, so we made **join codes**. Laptops on the same Wi-Fi share the start of their IP address, so the host only shows the end, like 144, and the joiner's game rebuilds the full address from its own IP. Point at code_to_ip We also rank network adapters so it always uses real Wi-Fi rather than something like a VirtualBox adapter.
>
> The key design decision is **authority**: who's in charge of what. **Your own astronaut** runs on your machine, so the controls feel instant with no lag. But anything **shared**, like money, loot pickups and the relics, is decided by the **host**.
>
> So we use one pattern everywhere: **the client requests, the host decides, the host broadcasts**. Point at request_collect / confirm If two players touch the same gold at the same moment, both ask the host, the host accepts the first, and tells everyone. Buying upgrades works the same way, so nobody can spend the same money twice.
>
> And a shortcut we're proud of: we **don't send the loot over the network at all**. Every machine generates the same 160 pieces from the **same random seed**, so only the pickups travel. Players

> who join late get a list of what's already been taken."

If you forget: host = server · ENet, port 9999, 8 players · join code = end of IP · own movement local · shared stuff via host · request → decide → broadcast · same seed → only pickups sent.

5. The communication mechanics Speaker D 5:15 – 6:45

On screen: `tethers.gd` (links), `player.gd` (_pull_ropes), `heavy_relic.gd` (midpoint).

> "Now the three mechanics that make you talk.
>
> **Darkness and the radar.** The whole world is darkened with a *CanvasModulate*, and each helmet is a small point light, so divers only see a few metres around them. The radar player's screen isn't affected by the darkness: it draws every piece of loot, every teammate and their oxygen, scaled down onto a circle. Information is split between players, so they have to share it out loud.
>
> **Tethers.** Point at Tethers.links Everyone who isn't on the radar is automatically roped to a teammate in pairs, and the odd one out makes a chain of three. Each player's own machine applies a spring force when the rope is stretched, but we **capped it below jetpack thrust**: if you keep pulling, the rope turns red, and at 135% of its length it **snaps and both of you black out**. The pairing is calculated the same way on every machine, so it needed no extra networking.
>
> **Heavy relics.** Point at the midpoint code They're worth $500 and only move when two players hold on. The host moves the relic toward the **midpoint** of its carriers, so if you pull in different directions, it goes nowhere.
>
> And when your air runs out, you don't die instantly: you get about **five seconds** of health, with red edges and an alarm, which is just long enough for a teammate to panic."

If you forget: darkness + lamps · radar sees all · ropes in pairs · spring capped below thrust · snap at 135% → both black out · relic needs 2, moves to midpoint · 5 s of health with no air.

6. Polish & testing Speaker D 6:45 – 7:30

On screen: `nebula.gdshader`, or the game with the background turning red.

> "For polish: the background is a **pixel-art nebula shader**, adapted from an MIT-licensed generator, and we made it change from calm blue to danger red the further you drift from home. There are particle effects for jetpacks, pickups and snaps, positional sound, and pixel-art sprites with eight different looks.
>
> To test multiplayer without needing four people every time, we wrote an **autopilot bot**. It collects loot, comes home and buys upgrades, so we could run a host and a client on one laptop and check every system from the logs."

If you forget: nebula shader (MIT), turns red with distance · particles, sound, sprites · autopilot bot for testing.

7. Challenges & close Speaker A 7:30 – 8:00

On screen: **the win screen with confetti**.

> "Our favourite bug: the **host never saw the win screen**, because the host updated the part count before checking whether we'd just won. A one-line flag fixed it.
>
> Next, we'd add internet play and meteors for the radar operator to warn about.
>
> Space Salvage turns talking into the game: **you can't win without your crew**. Thank you, and we're happy to take questions."

If you forget: win-screen bug · next: internet play, meteors · "you can't win without your crew" · thank you.

2 Demo run sheet

Time	Projector laptop: host + diver (Speaker B)	Player 2: radar · Player 3: diver	What the audience should notice
0:45	Click **Host Game**; read the join code aloud	Both type the code, click **Join Game**	Joining with just 3 numbers
0:55	Press **R** to show the radar, then **R** again	Player 2 presses **R** (takes the radar)	Gravity inside; the radar sees everything; the two divers get roped together
1:10	Walk out the airlock with Player 3; follow the call-outs	Player 2 calls out gold and air % **loudly**; Player 3 dives	The dark; the helmet lamp; the rope; gold sparkles + "+$" popups
1:40	Both divers press **E** on the relic and haul it home	Player 2 guides them to the cyan diamond	"1/2 → 2/2"; it only moves with both
2:05	"And if we stop listening…" Fly away from Player 3	Player 3 flies the opposite way	Red rope, "TETHER STRAINING", snap → both black out
2:15	Back inside the station, press **3** five times (Pod 5/5)	—	Camera zooms out, pod rumbles and launches, confetti
2:30	Leave the win screen up	Hands off	Hand over to Speaker C

Backup plans

If this happens…	Do this
Join hangs on "Connecting…"	Try the longer backup code shown next to it. Still stuck: everyone onto the phone hotspot. Still stuck: solo demo (below). It gives up after 10 s by itself.
No time for networking at all	Solo: host alone, you're roped to the station. Show the dark, press R inside for the radar, then press 3 five times for the pod launch (you started with `--money=6000`).
The relic is too far away	Skip it and describe it: "relics worth $500 need two carriers." Do the rope snap instead.
Someone blacks out early by accident	Lean into it: "and that's what happens when you don't listen to the radar."
No sound	Check the projector laptop's volume and that the right output device is chosen. Carry on: the visuals tell the story.
Running long	Skip the relic; go straight from the radar call-outs to the snap and pod launch.

> **The money shot:** the rope snap gets the biggest reaction. Because the projector laptop is a diver, the audience sees the dark, the rope going red and the snap. The radar part comes across through the operator's call-outs, so they need to be **loud**.

3 Questions (2 minutes)

Strategy: two minutes is about **4–5 questions**. Answer in **~20 seconds**: the short answer first, then one detail if they look interested. The person who presented that topic answers it. If you don't know: "*Good question. We*

didn't get to that in the hackathon, but we'd approach it by…"

How does the multiplayer work? C

Short: One player hosts over ENet and the others join with a 3-number code. You control your own astronaut locally, and the host decides anything shared.

More: Godot's MultiplayerSpawner copies players to every machine, and a MultiplayerSynchronizer streams position, oxygen and health every frame.

How do you stop two people grabbing the same loot, or cheating? C

Short: Pickups and purchases are requests to the host, and the host accepts the first one. It's co-op among friends, so movement is trusted for speed.

More: `request_collect` → host checks the loot is still active → `confirm` broadcast to everyone.

How does the game fit the theme? A

Short: Information is split between players: divers are blind, the radar player sees everything, ropes force pairs to agree, and relics need two people pulling the same way.

How do the tethers work technically? D

Short: Automatic pairs; a spring force capped below thrust; it snaps at 135% of the rope length and both players black out.

More: the pairing is computed identically on every machine from synced data, so it needs no extra network messages.

How do you sync 160 pieces of loot? C

Short: We don't. Every machine generates them from the same random seed, so only the pickups are sent.

What did you make vs. use? A

Short: All the gameplay code is ours. The sprites, sounds and music are free CC0 assets, and the nebula shader is adapted from an MIT-licensed generator. All are credited in the README.

What was the hardest part? C

Short: Networking edge cases, like people joining mid-game and the host's own win screen, and getting join codes to pick the right network adapter.

Can it be played over the internet? C

Short: Right now it's same-Wi-Fi. Internet play needs port forwarding, a relay server or Steam networking. That's our next step.

What would you add with more time? A

Short: Internet play, meteor hazards for the radar player to warn about, more relic types, and balancing from playtests.

How did you test it? D

Short: We wrote an autopilot bot and ran host and client instances on one laptop, including with no window, then checked the logs and took automatic screenshots.

Why Godot? C

Short: It's free, fast for 2D, and has multiplayer built in (spawning, syncing and remote function calls), which made an 8-player game possible in a hackathon.

4 Speaker cue cards

Print this page, cut it into four, and hold your card while presenting.

Speaker A — Hook (0:00) + Close (7:30)

- "Space Salvage: nobody can succeed alone."
- *Dave the Diver* in space, side-on like *Terraria*.
- Drift out · grab gold (more value further out) · get home before the air runs out.
- Sell → upgrade → repair the escape pod.
- Theme = communication: **radar · ropes · relics**.
- "Let us show you." → hand to B.
- **Close:** win-screen bug (one-line fix) · next: internet play, meteors.
- "You can't win without your crew. Thank you, questions?"

Speaker B — Live demo (0:45 – 2:30)

- Host → read the join code aloud → they join.
- Gravity inside → **R** to show the radar → **R** again; Player 2 takes it.
- Dive: "it's dark, I'm roped, I rely on the voice from the station."
- Follow Player 2's call-outs; grab gold.
- Relic: "$500, needs both of them, goes to the point between them."
- Snap: "and if we stop listening…" → both black out.
- Inside: press **3** ×5 ("we gave ourselves money for the ending") → pod launch.
- Backup: solo demo; hotspot; skip the relic if running long.

Speaker C — Built (2:30) + Multiplayer (3:30)

- Godot 4.7 · GDScript · ~2,600 lines · 15 scripts + 1 shader.
- Autoloads: **NetworkManager** (connect) · **GameState** (money).
- `world.gd` builds everything · UI in code → no git conflicts.
- Host = server · ENet · UDP 9999 · 8 players.
- Join code = the end of the host's IP; rebuilt from your own IP.
- Own astronaut = local (instant) · shared stuff = host.
- **Request → host decides → broadcast** (loot, buying).
- Same seed → 160 loot on every machine → only pickups are sent.

Speaker D — Mechanics (5:15) + Polish (6:45)

- Darkness: CanvasModulate + helmet point lights.
- Radar: sees all loot + teammates' air → info is split.
- Tethers: auto pairs · spring capped below thrust · snap at 135% → both black out.
- Relics: $500 · 2 carriers · move to the midpoint.
- No air → ~5 s of health, red edges, alarm.
- Nebula shader (MIT) → turns red further from home.
- Particles · positional sound · 8 sprite looks.
- Autopilot bot → tested on one laptop.

> **If everyone remembers only three lines:** (1) "Communication is the mechanic: blind divers, a radar operator, ropes and two-person relics." (2) "Your own astronaut runs locally for instant controls; the host decides anything shared." (3) "Loot comes from a shared seed, so we only send pickups, and we tested it all with our own bots."